Unité 1: Blockly avancé - bonnes pratiques de programmation robotique

Site:	NiryoAcademy	Imprimé par:	Theo Bernicot
Cours:	De Blockly à Python avec le robot Ned2	Date:	vendredi 16 janvier 2026, 15:30
Livre:	Unité 1: Blockly avancé - bonnes pratiques de programmation robotique		

Table des matières

Objectifs pédagogiques

Commentaires

Séquences

Variables

Boucles

Conditions

Fonctions

Listes

Objectifs pédagogiques

- Comprendre et appliquer les bonnes pratiques de programmation avec Blockly.
- Utiliser commentaires, séquences, variables, boucles, conditions, fonctions et événements.
- Préparer la transition vers Python en consolidant la logique et la réutilisabilité du code.

En pratique:

Les cas pratiques seront effectués depuis l'outil de simulation de NiryoStudio. Vous pouvez les répliquer bien entendu sur votre robot physique.

Si besoin, vous pouvez revoir le tutoriel suivant pour apprendre à déplacer et à enregistrer des positions du robot sur Blockly.

Commentaires

Concept :

Les commentaires expliquent **pourquoi** on fait quelque chose, pas **ce qu'on fait**. Ils sont une bonne pratique en programmation car ils permettent de documenter votre code, de le rendre plus lisible pour pouvoir le partager et permettre de mieux le comprendre.

Voici le bloc ***Commentaire*** dans Blockly:

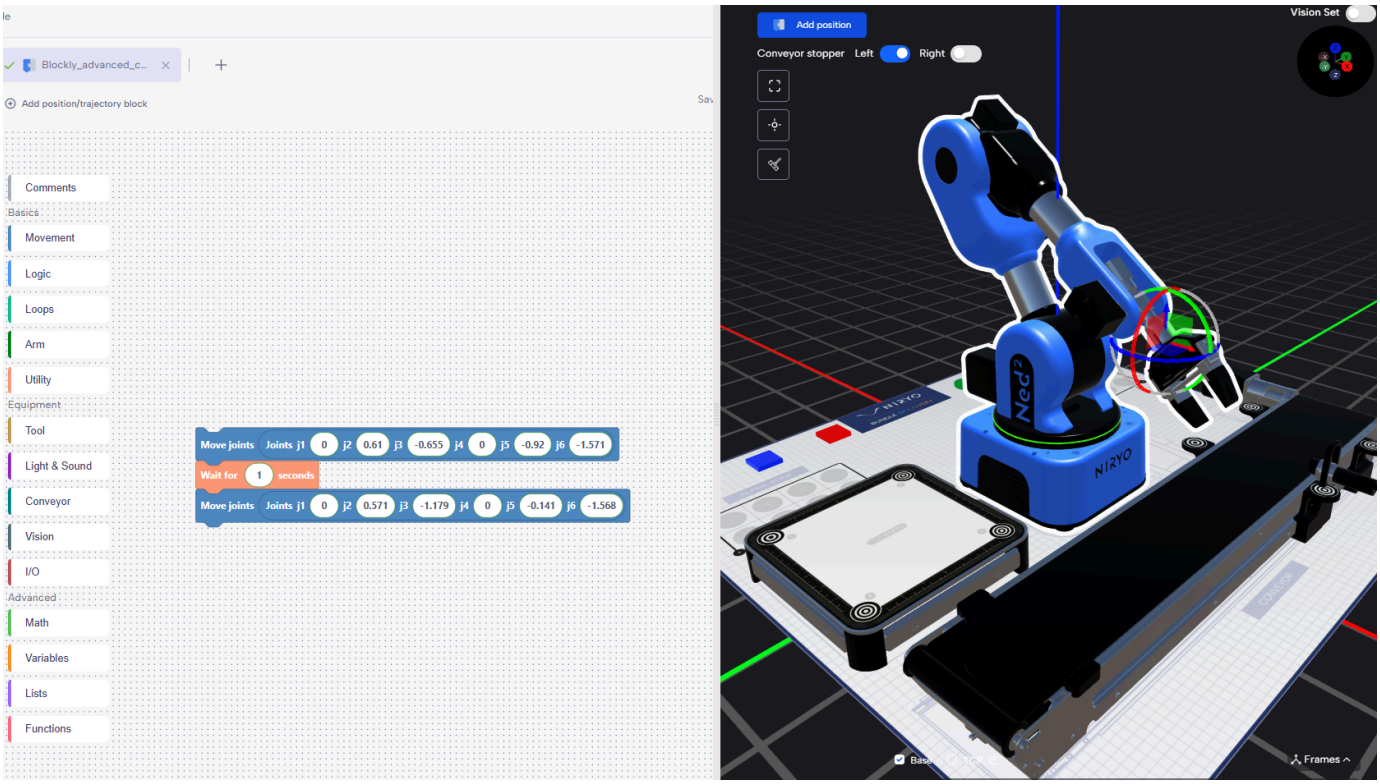
Cas pratique:

Objectif: Programmer le robot pour saluer en ajoutant les commentaires pour documenter le code.

Etape 1:

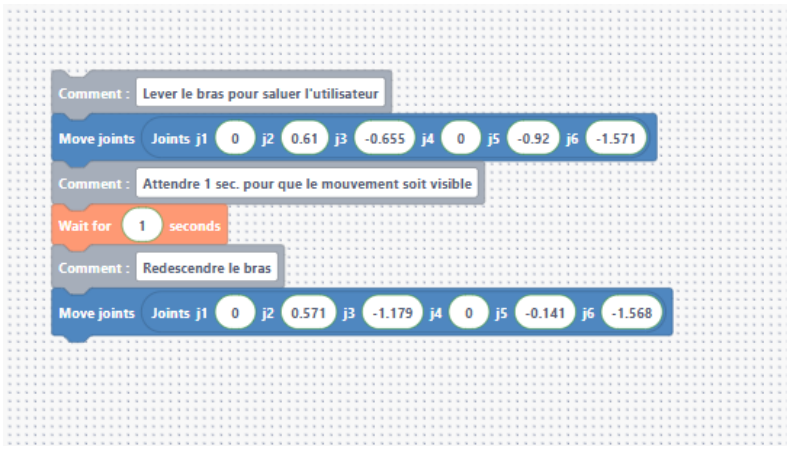
Nous allons d'abord prendre les positions auxquelles le robot doit se déplacer pour effectuer son mouvement de salut.

Il faut prendre deux positions: la première en haut, puis une seconde en bas. On peut ajouter un bloc ***wait*** pour accentuer le mouvement.



Etape 2:

Avant chaque bloc, nous pouvons maintenant ajouter des blocs ***Commentaires*** pour expliquer pourquoi on fait ce mouvement:



Télécharger un exemple de solution:

🔗 Lien de téléchargement

💡 Vous pouvez importer ce programme directement dans NiryoStudio depuis Librairie > Import > fichier .json

Séquences

Concept

Une séquence représente un enchaînement d'instructions qui s'exécutent **dans l'ordre**.

Il est important en amont d'une séquence de bien planifier les instructions pour éviter les erreurs.

Cas pratique:

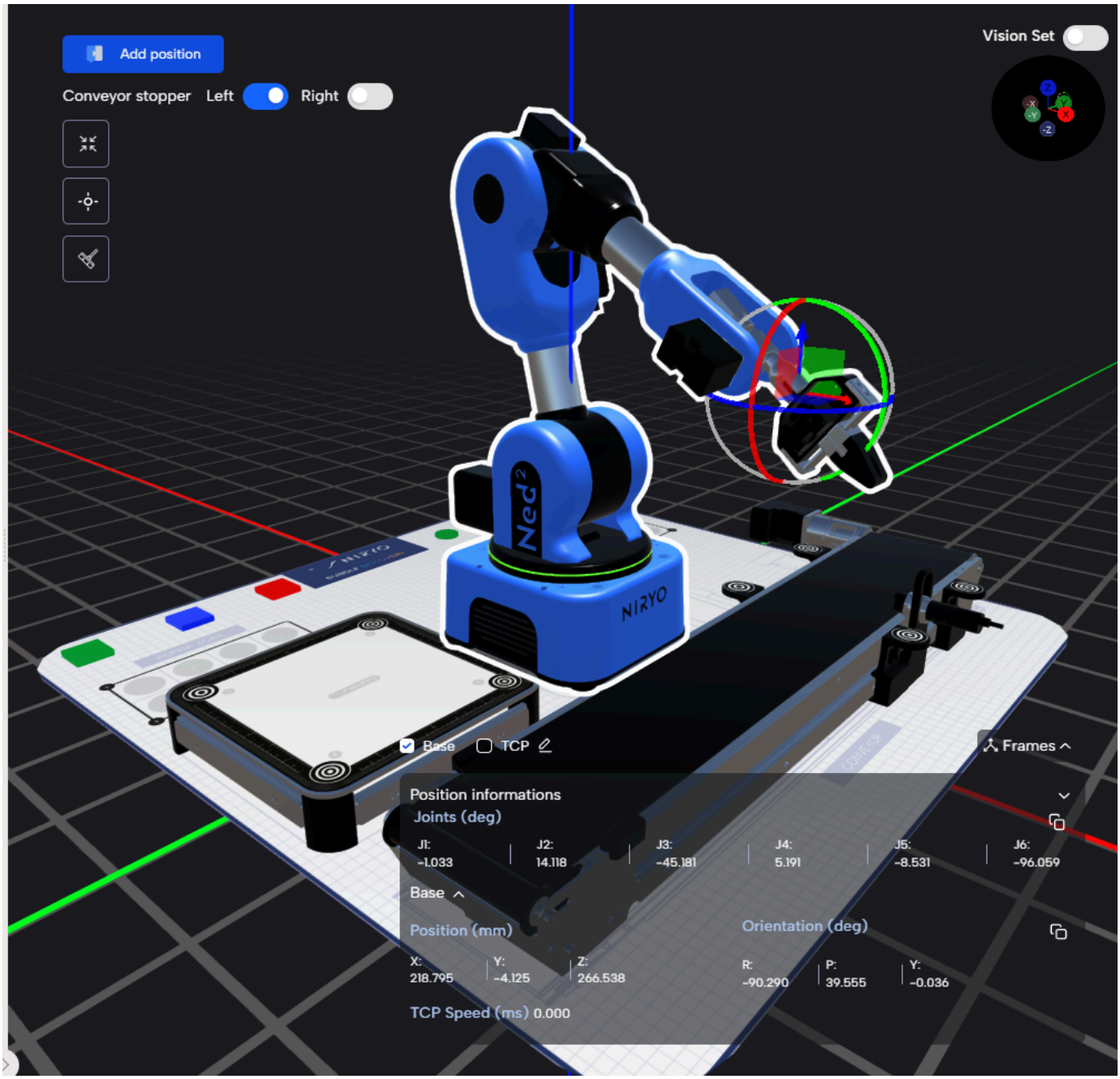
Nous allons réaliser une séquence classique en robotique: le pick & place. Il s'agit d'une séquence type d'instructions:

1. Déplacer le robot vers une position
2. Fermer la pince (ou un autre outil)
3. Déplacer le robot vers une autre position
4. Ouvrir la pince

pour pouvoir déplacer un objet.

Ce type de séquences est très fréquent pour l'automatisation de tâches industrielles.

Pour déplacer ici le jeton vert depuis sa zone de réserve jusqu'à la zone de dépose "Plateau vision" comment feriez-vous? Dans votre programme Blockly, combien de blocs position ajouteriez-vous?



La réponse simple pourrait être de dire qu'il faut:

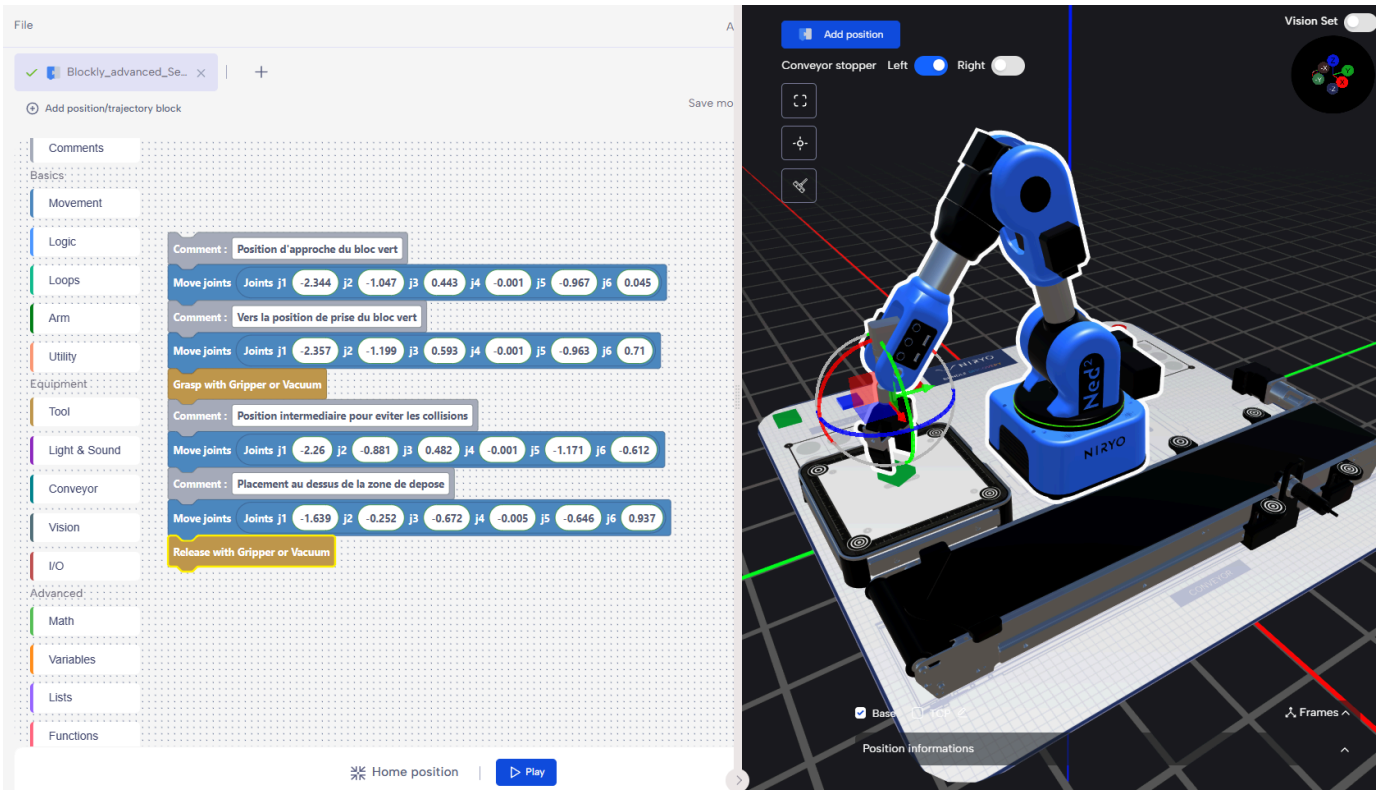
- 1 bloc position: pour s'approcher du bloc vert
- 1 bloc fermer le gripper
- 1 bloc position: pour être au-dessus de la zone de dépose
- 1 bloc pour ouvrir le gripper.

Attention: si vous exécutez votre programme ainsi, il entrera en collision avec le plateau vision qui est légèrement surélevé. C'est pourquoi il est important de **planifier vos séquences** avant de commencer à programmer.

En pratique, nous conseillons toujours d'ajouter des **positions intermédiaires**:

- Pour prendre le bloc vert: on utilise une position d'approche (au-dessus du bloc) avant d'ajouter la position pour attraper le bloc
- Idem pour la dépose: on utilise une position intermédiaire (entre la prise et la dépose) pour éviter toute collision sur la trajectoire du robot.

Voici un exemple de solution:



Télécharger un exemple de solution:

🔗 Lien de téléchargement

💡 Vous pouvez importer ce programme directement dans NiryoStudio depuis Librairie > Import > fichier .json

Variables

Concept

Les variables stockent des valeurs qui peuvent changer. En robotique, les variables peuvent être utiliser pour stocker des positions.

Le but est d'éviter que le code soit rigide et répétitif et de faciliter les modifications.

Cas pratique:

Nous allons reprendre la séquence de pick & place que nous avons créée dans le chapitre "**Séquences**" et apporter des étapes supplémentaires.

Partie 1:

Maintenant, au lieu de déposer toutes les pièces vertes sur la plaque de vision, je vous demande de:

- Déposer 1 jeton sur la plaque vision (séquence inchangée)
- Déposer 1 jeton sur le convoyeur
- Déposer 1 jeton sur la "Alpha zone".

Pour procéder, vous allez avoir besoin de réutiliser plusieurs fois deux positions:

- La position d'approche du bloc vert
- La position de prise du bloc vert

🚫 Vous allez donc, copier et coller ces blocs à d'autres endroits du code.
C'est fastidieux, et surtout c'est propice aux erreurs - il ne faut pas mélanger les deux positions quand vous les réutilisez...

Partie 2:

Enfin, je vous demande maintenant de faire la même séquence (1 jeton sur la plaque vision, 1 autre sur le convoyeur et 1 dernier sur la "Alpha zone") mais avec les **jetons bleus.**

🚫Pour procéder, vous allez reprendre toute votre séquence, la copier et modifier uniquement les blocs d'approche et de saisie du bloc vert, pour les modifier avec les positions d'approche et de saisie du bloc rouge.
C'est long à faire, propices aux erreurs et votre code devient très vite illisible.

C'est là que les variables deviennent intéressantes. Au lieu de "coder en dur" les positions de votre robot, vous allez :

1. Créer des blocs "***variables***"
2. Stocker chaque position dans ces variables
3. Appeler ces variables dans votre programme

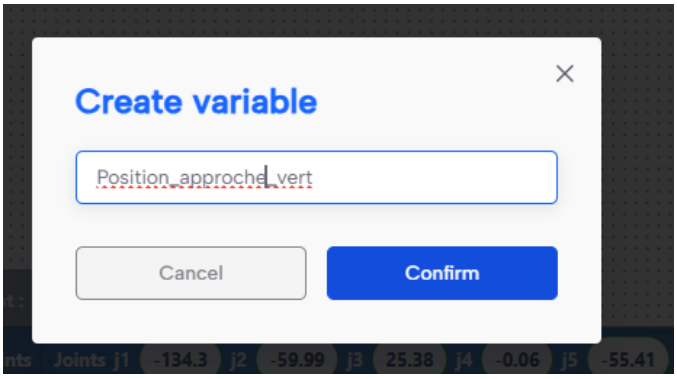
Résolution de la partie 1:

Je vous montre comment faire:

1. Dans Blockly > Advanced > Variables > Create Variable



2. Je la nomme:

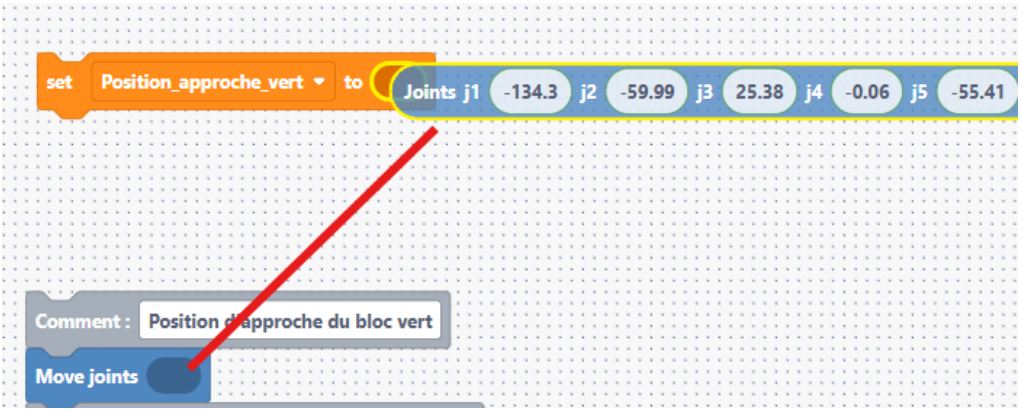


3. De nouveaux blocs apparaissent:

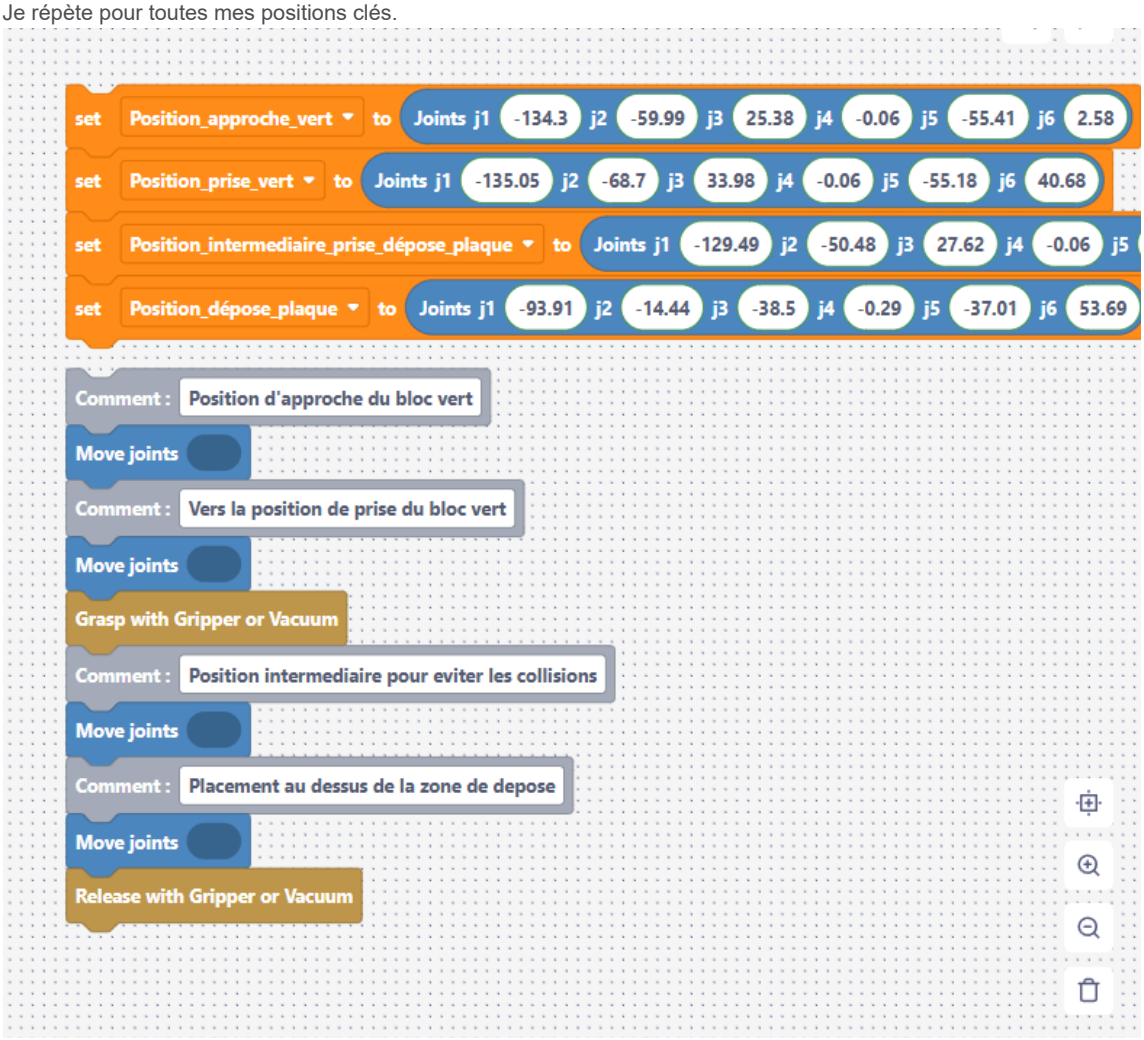


1. Le premier "Set...." permet de **définir la valeur de la variable qui est sélectionnée**
2. Le deuxième "Change..." permet de **modifier la valeur de la variable par la donnée entrée**
3. La troisième est la variable elle même, qui peut donc être utilisée (appelée) dans d'autres fonctions Blockly (par ex. dans un bloc de mouvement)

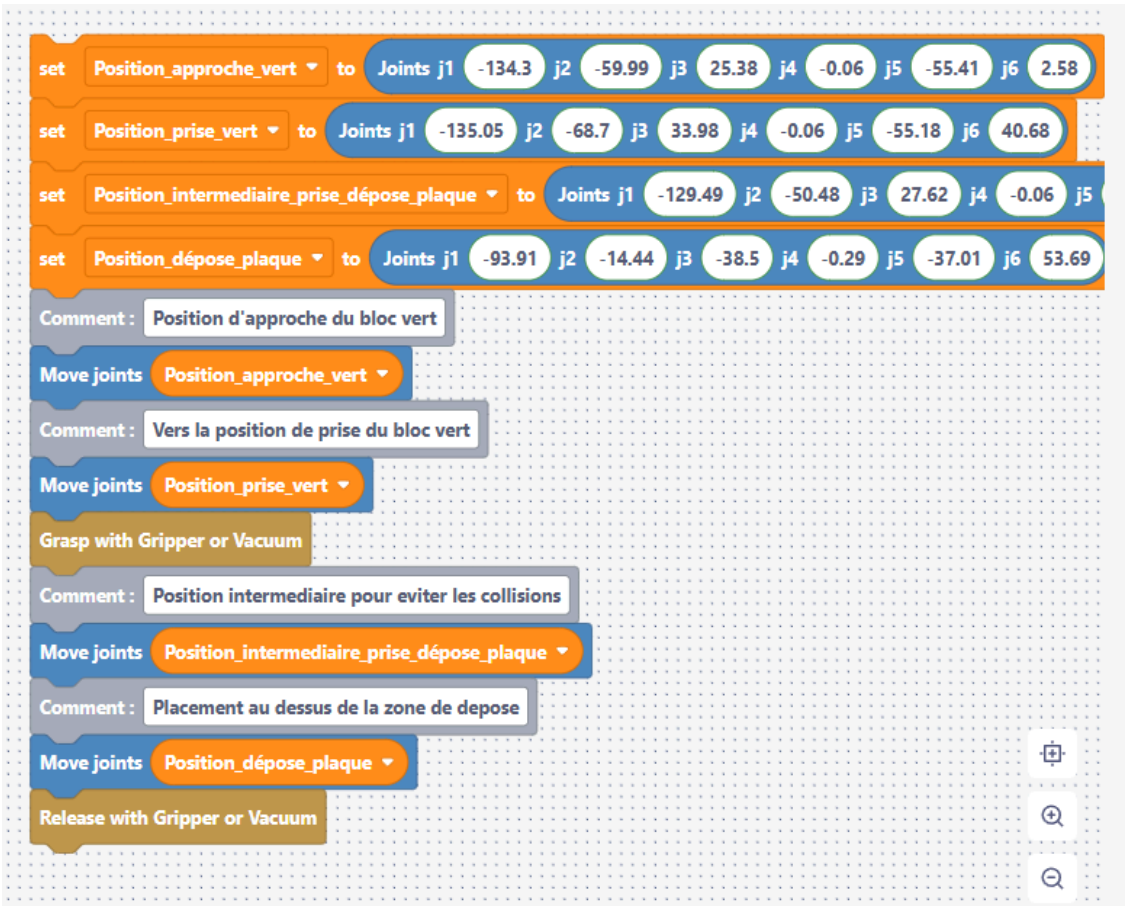
4. J'utilise le premier bloc (pour définir ma variable) et lui donne les valeurs de position que j'ai enregistrées lorsque j'ai défini ma position.



5. Je répète pour toutes mes positions clés.

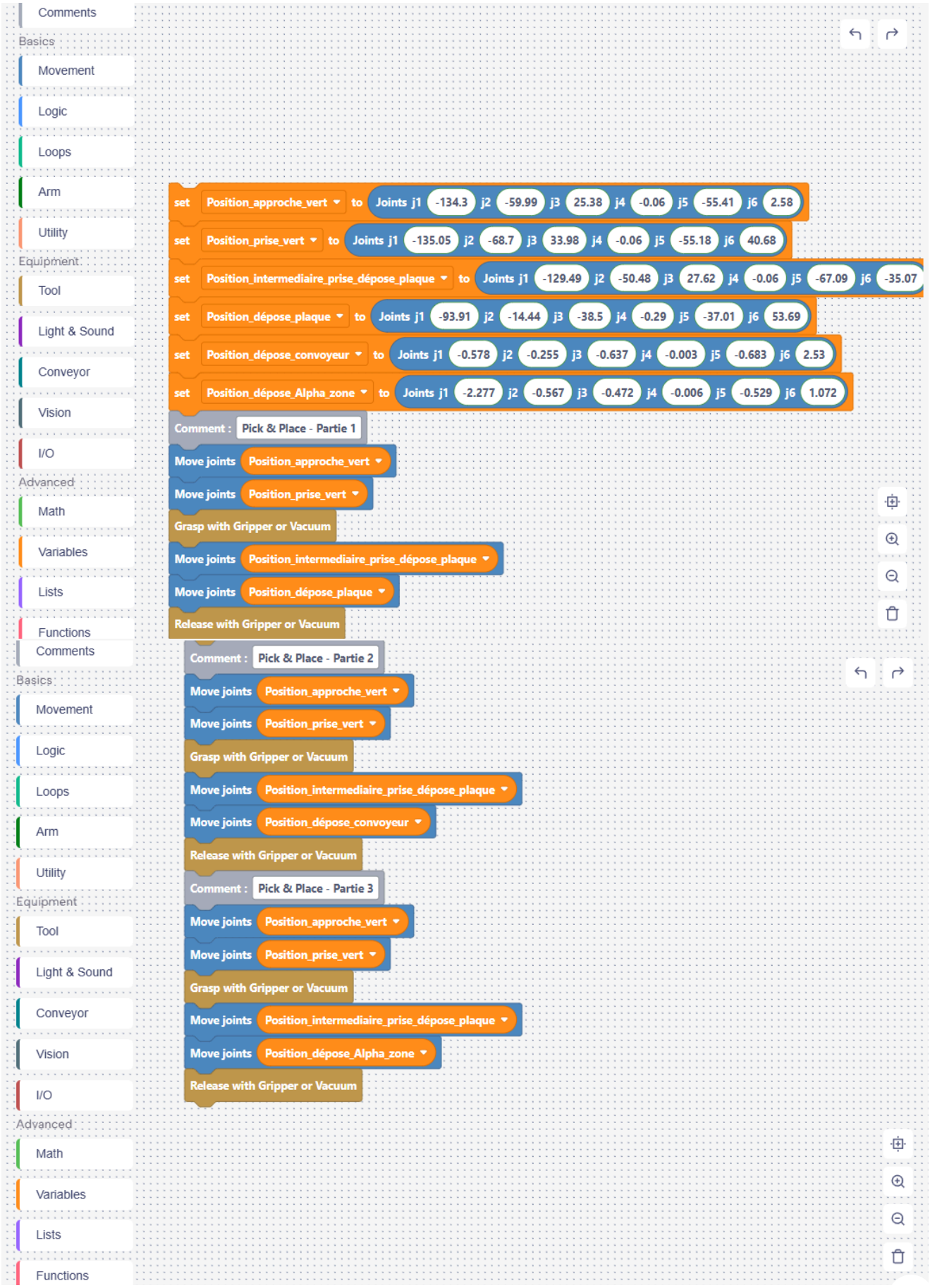


6. Je peux maintenant finaliser ma séquence 1.



7. Et je peux compléter la séquence facilement avec la partie 2 et la partie 3.
    1. Je crée les variables pour la zone de dépose convoyeur et j'y stocke les positions prises avec le robot
    2. Je crée les variables pour la zone de dépose "Alpha zone" et j'y stocke les positions prises avec le robot

8. Je peux maintenant copier ma séquence "Pick & Place" pour la répéter 3 fois, en modifiant rapidement et simplement les zones de dépose.



💡 Ici j'ai modifié mes commentaires pour mieux identifier mes différentes séquences de pick & place.

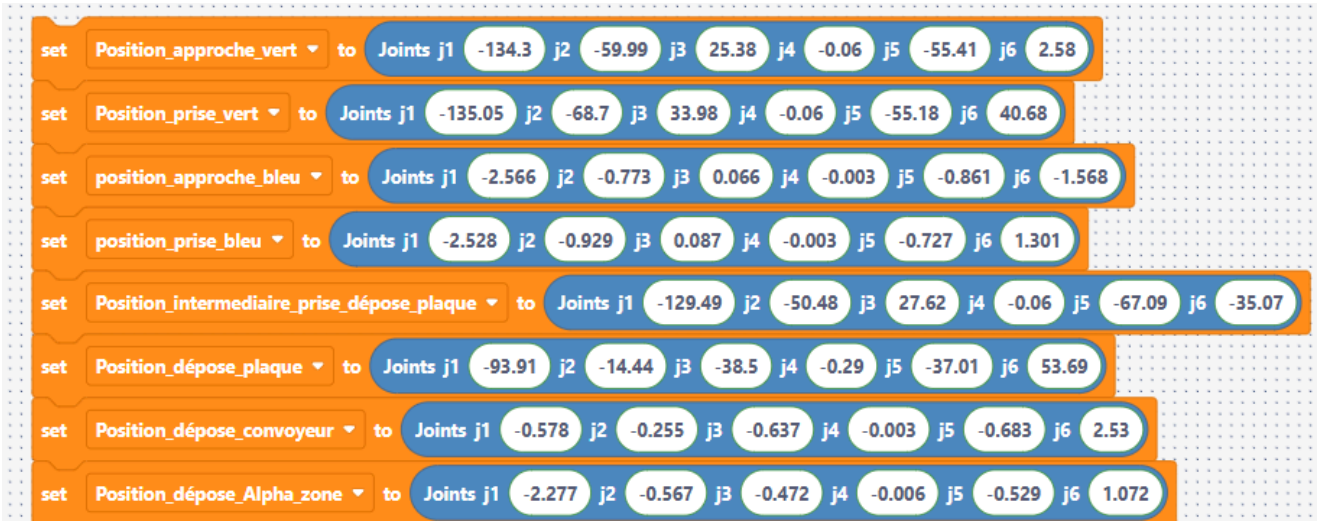
Résolution de la partie 2:

Maintenant, on va plutôt saisir les jetons bleus. Comment feriez-vous, en utilisant les variables?

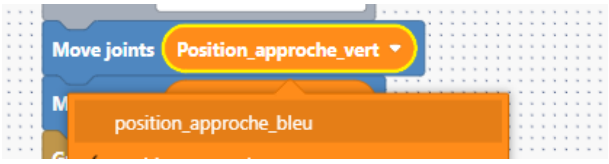
1. On va d'abord créer les variables **position_approche_bleu** et **position_prise_bleu**



2. Maintenant je vais pouvoir y stocker la position prise avec l'aide du robot



3. Et je peux simplement modifier mon programme précédent en modifiant la **variable** dans le menu déroulant:



4. Voici le résultat:



Les variables sont très puissantes pour:

- Modifier rapidement des positions et ne pas avoir à retoucher tout votre programme
- Réutiliser la même position plusieurs fois dans un programme, sans risque d'erreur
- Améliorer la lisibilité de vos programmes.

Télécharger un exemple de solution:

🔗 Lien de téléchargement

💡 Vous pouvez importer ce programme directement dans NiryoStudio depuis Librairie > Import > fichier .json

Boucles

Concept

Une boucle permet de répéter des instructions selon des critères définis par l'utilisateur.

L'objectif des boucles est de réduire la répétition, améliorer la maintenance du code.

Cas pratique:

Nous reprenons la séquence créée la séance précédente. *Si vous ne l'avez pas faite, vous pouvez tout de même consulter cette partie et télécharger le programme pour partir de cette base.*

Partie 1

Je souhaite maintenant déposer 3 jetons bleus dans la zone de dépose "plaque de vision".

🚫 Pour procéder, vous allez donc

- Sélectionnez les blocs de la séquence "Pick & Place - Partie 1"
- Les copier
- Les coller à la suite de cette séquence, 2 fois.

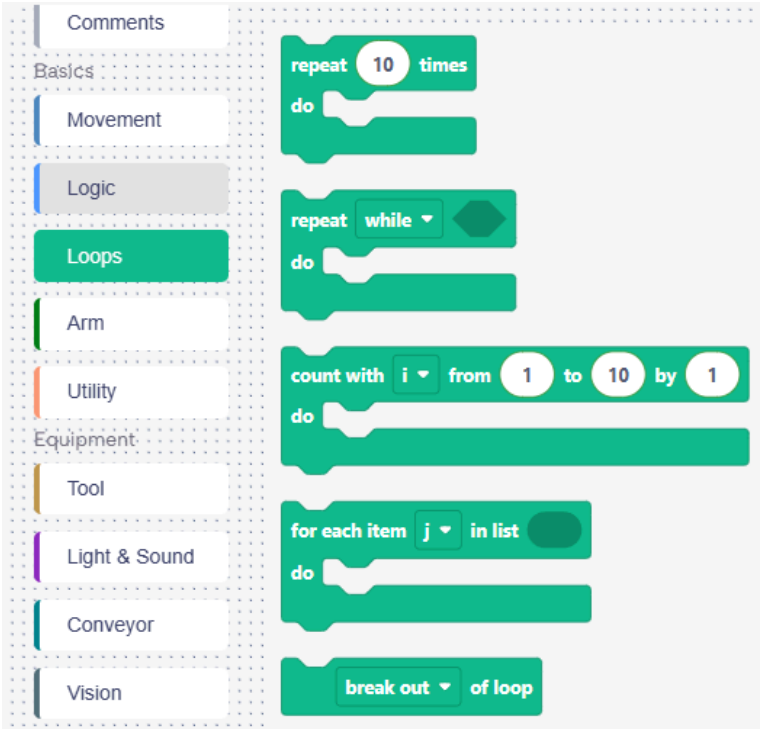


Ici, on ne demande que 3x répétitions. Ca semble donc faisable. Mais imaginez qu'on demande d'en prendre, 10, 20... Imaginez la longueur de votre code et la possibilité d'erreurs.

Résolution de la partie 1

Pour répéter des instructions simplement, on utilise les ***Boucles***.

1. Dans Blockly > Basics > Loops vous trouverez les blocs correspondants aux boucles

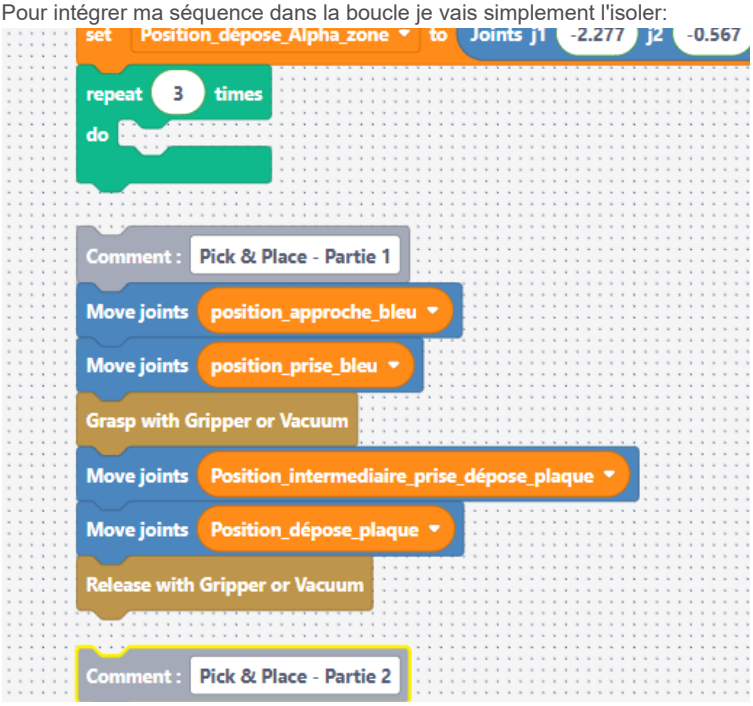


Il existe plusieurs types de Boucles. Nous allons utiliser la première: ***repeat xxx times .. do***

2. Son fonctionnement?
    1. Ce bloc entoure d'autres blocs d'instructions. Ce sont blocs là qui seront répétés.
    2. Vous pouvez modifier la donnée numéraire pour indiquer le nombre de répétition. Dans notre cas, on indique "3"



3. Pour intégrer ma séquence dans la boucle je vais simplement l'isoler:



4. Et la saisir pour l'intégrer dans le bloc boucle



5. Résultat:



💡 Je peux maintenant modifier le nombre de répétitions rapidement et ajuster mon programme sans le rallonger ou le complexifier.

Télécharger un exemple de solution:

🔗 Lien de téléchargement

💡 Vous pouvez importer ce programme directement dans NiryoStudio depuis Librairie > Import > fichier .json

---

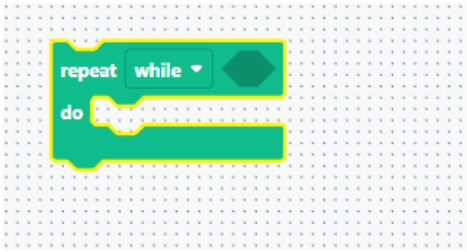
Partie 2

Je souhaite maintenant déposer des jetons bleus dans la zone de dépose "plaque de vision" **en continu**.

On peut imaginer par exemple, que vous êtes dans un entrepôt logistique et votre robot doit prendre en continu des colis qui arrivent depuis une rampe. Le nombre de colis qui arrivent est inconnu, on sait juste que le flux est continu. Si le robot doit s'arrêter, c'est uniquement si on lui en donne l'instruction.

Résolution de la partie 2

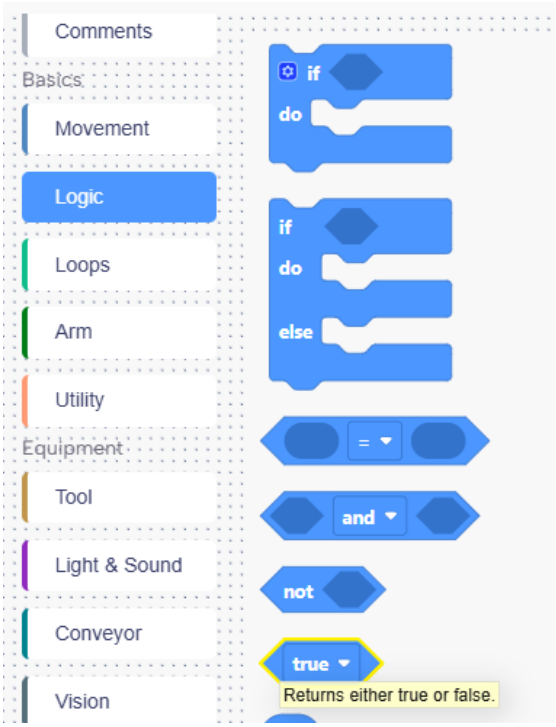
Dans ce cas, j'utilise la boucle:



Elle peut se paramétrer de deux manières:

- ***While***: vérifie la condition **avant** d'exécuter le corps de la boucle.
  - Si la condition est fausse dès le départ, le corps de la boucle ne s'exécute **jamais**.
  - Après chaque exécution complète du corps de boucle, la condition est vérifiée à nouveau.
  - Si elle est encore vraie, la boucle recommence.
- ***until*** : vérifie la condition **après** avoir exécuté le corps de la boucle.
  - Même si la condition est fausse au départ, le corps de la boucle s'exécute **au moins une fois**.
  - Une fois le corps exécuté, la condition est testée.
  - Tant qu'elle reste fausse, la boucle continue.

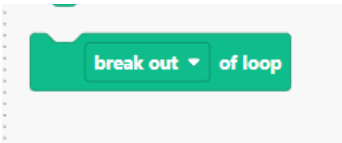
Ici, je vais lui donner une condition "***True***"



Cela signifie: répète la boucle, sans interruptions.

Résultat:



Pour l'interrompre, je dois utiliser un bloc: 

Télécharger un exemple de solution:

🔗 Lien de téléchargement

💡 Vous pouvez importer ce programme directement dans NiryoStudio depuis Librairie > Import > fichier .json

---

Partie 3

Reprenons l'exemple de mon robot dans un entrepôt logistique. Cette fois ci, le robot est positionné à côté d'un convoyeur qui lui apporte des colis. Je souhaite:

- Alimenter le convoyeur avec des colis
- Détecter si un colis passe sur le convoyeur
- Stopper le convoyeur si un colis est détecté
- Saisir le colis
- Le déposer sur une zone de dépose (plaque vision)

Pour effectuer ce cas d'usage, dans la robotique industrielle, nous utilisons des capteurs. Vous en avez aussi un sur votre Discovery Bundle sur le **convoyeur**: c'est le **capteur infrarouge**. Ce capteur est branché au niveau de l'entrée digitale n°5 du robot (DI-5).
Il fonctionne ainsi:

- Il envoie "HIGH" au robot, lorsqu'aucun objet n'est pas détecté
- Il envoie "LOW" au robot, lorsqu'un objet est détecté

Je peux donc dire à mon robot:

1. lance le convoyeur **jusqu'à ce que** le capteur détecte un objet.
2. Puis stoppe le convoyeur.
3. Robot: saisit l'objet

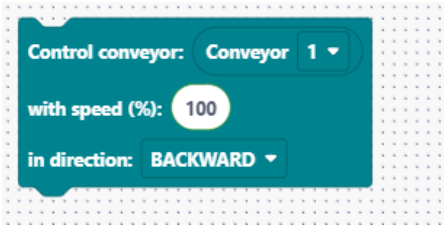
Résolution de la partie 3

Voici comment faire:

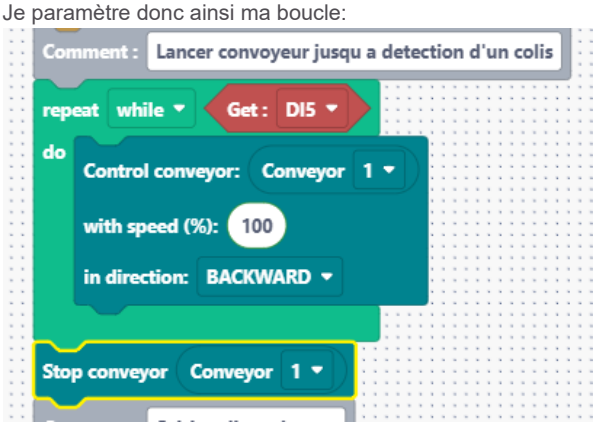
1. J'alimente avec mon robot la partie "amont" du convoyeur avec un jeton bleu (couleur au choix).



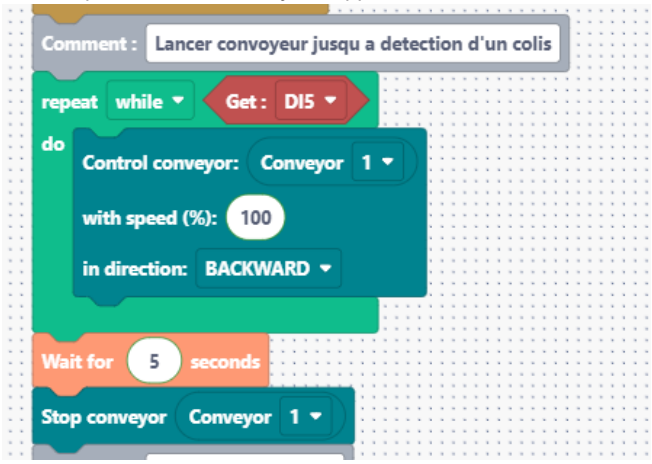
2. Puis je lance le convoyeur



3. Mais je veux qu'il s'arrête si le capteur détecte un objet. Je vais donc **l'insérer dans un bloc "Boucle" jusqu'à ce que le capteur DI-5 renvoie "LOW"**.
   Je paramètre donc ainsi ma boucle:



4. J'ajoute un bloc ***Wait***. Ce bloc permet de temporiser l'instruction qui le suit.
   Ici je vais demander à attendre avant de stopper le convoyeur pendant 5 secondes, pour laisser le temps au jeton bleu d'atteindre la zone de prise dans le "conveyor stopper".



5. Puis je peux finaliser la séquence et positionner mon robot dans la zone de prise pour ensuite aller déposer le jeton saisi dans la zone de dépose "plaque vision".



Télécharger un exemple de solution:

🔗 Lien de téléchargement

💡 Vous pouvez importer ce programme directement dans NiryoStudio depuis Librairie > Import > fichier .json

Conditions

Concept

Les **conditions** permettent au robot de prendre des décisions selon des conditions données.

Les conditions sont les outils parfaits pour gérer des situations variables et inattendues.

Cas pratique:

Nous allons reprendre la séquence de pick & place que nous avons créée dans le chapitre "**Boucles**" et apporter des conditions supplémentaires.

Partie 1:

Je souhaite désormais préparer une commande qui remplit les conditions suivantes:

- Saisir 1 colis bleu
- Saisir 2 colis verts

🚫Pour procéder, vous allez pouvoir utiliser une boucle qui répète la séquence "Pick & place Jeton Bleu" et insérer manuellement le nombre de répétition qui correspond à la commande (ici 3). Idem pour la boucle de la séquence "Pick & Place Jeton Vert", en insérant le nombre de colis à expédier (ici 2).

Mais que se passe-t-il si maintenant on souhaite simplement modifier le nombre de colis d'une couleur à préparer? Imaginons que votre programme soit encore plus long, plus complexe: dans l'idéal on aimerait avoir 1 emplacement en début de programme où l'on donne les paramètres de la commande à préparer.

Résolution de la partie 1:

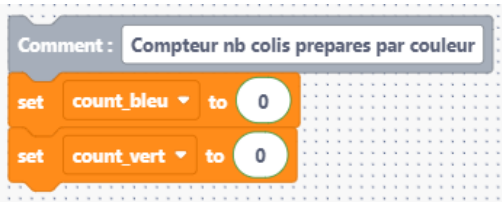
On peut ici utiliser les notions abordées précédemment: les boucles, les variables et les ***conditions***.

En synthèse je vais définir:

- Les variables qui définissent le nombre de colis bleu et vert à préparer dans ma commande



- Les variables qui me permettent de compter le nombre de colis qui sont préparés. Ces variables permettront au robot de vérifier à la fin de chaque séquence de pick & place d'une couleur s'il a préparé le bon nombre de colis.
  En début de programme, aucun colis n'est encore préparé, on met les compteurs à 0.



Je peux maintenant programmer ma préparation de commande:

1. J'utilise la boucle ***repeat until... condition***



J'y insère une **Condition** qui dit: compare la valeur de "count_bleu" et la valeur de "cde_bleu". Répète tant que ces deux valeurs ne sont pas **égales.**
Pourquoi?: on demande ici au robot de répéter les instructions dans la boucle, **tant que** le compteur bleu n'est pas égal au nombre de colis à préparer pour la commande.

2. J'insère maintenant ma séquence "Pick & Place Jetons bleus"



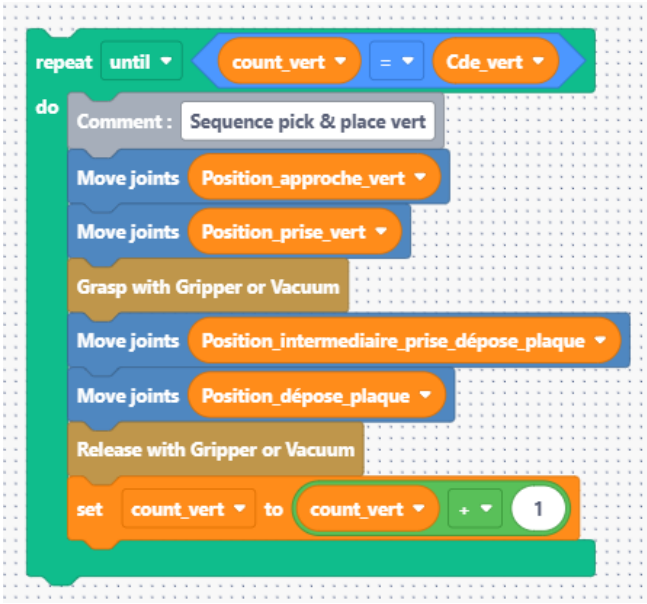
3. A la fin de la première itération de la boucle, j'ai préparé 1 colis. Il faut donc que je modifie la valeur de "count_bleu" pour faire le bon décompte. J'ajoute donc un bloc pour modifier la valeur de "count_bleu" en fin de séquence.



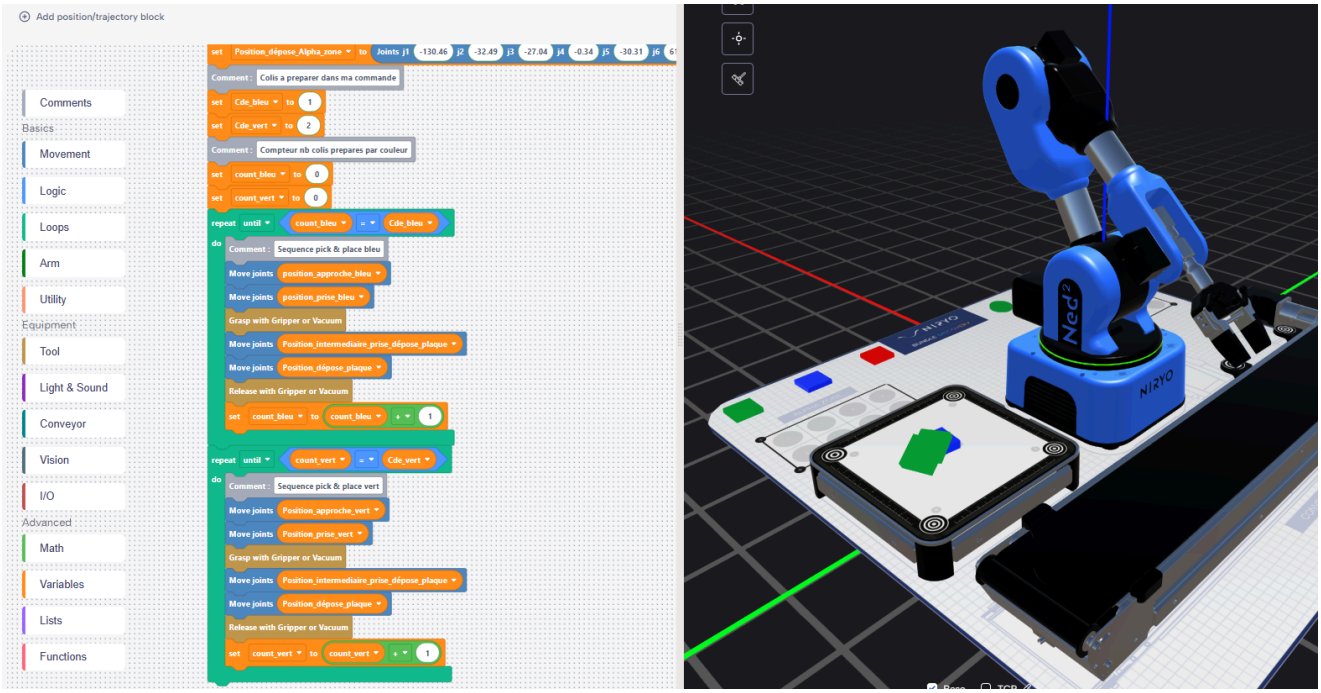
Ainsi, à la fin de chaque itération de la boucle, je sais que j'ai préparé le "nombre de colis préparés" + 1 nouveau colis.

4. Résultat: ma boucle se répète, tant que je n'ai pas préparé exactement le bon nombre de colis dans ma commande.

5. On fait la même chose pour les colis verts:



6. Résultat:



Télécharger un exemple de solution:

🔗 Lien de téléchargement

💡 Vous pouvez importer ce programme directement dans NiryoStudio depuis Librairie > Import > fichier .json

---

Partie 2:

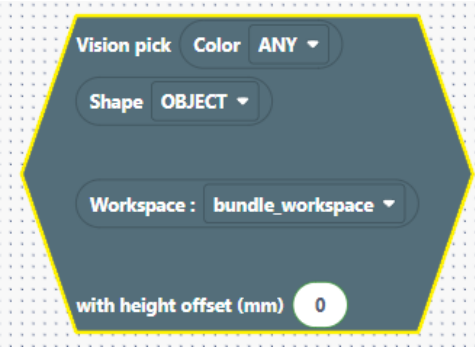
Mon robot est désormais placé en bord d'un convoyeur et doit trier des pièces. Je souhaite qu'il place:

- Les jetons bleus dans la zone "plaque vision"
- Les jetons verts dans la zone "Alpha zone"

Pour effectuer cette opération nous allons utiliser la **vision du robot NED2**.

Comment fonctionne la vision?

Le robot Ned2 peut être équipé d'une caméra. Le flux vidéo de cette caméra peut être interprété dans NiryoStudio et un algorithme de **détection de couleurs et de forme** est inclus dans le logiciel. Des blocs Blockly ***"vision"*** sont préformés pour faire des ***vision pick***:



- Workspace: je demande au robot d'identifier des pièces qui sont dans un espace de travail enregistré.
  Dans la simulation, ces espaces de travail sont prédéfinis. Il y a le workspace de la plaque de vision et celui du convoyeur.
  Sur un robot réel, il faut définir manuellement ces workspaces. Vous pouvez vous référez à la documentation suivante.
- Color <ANY> et Shape <OBJECT>: c'est ici que je peux donner les caractéristiques des objets à saisir.
  Sans modification: le robot va saisir de manière aléatoire un objet.
  Modification "Color": on peut choisir parmi les 3 couleurs de base dans le menu déroulant
  Modification "Object": on peut choisir cercle ou carré dans le menu déroulant

A l'issue du bloc, le "vision pick" s'effectue: c'est à dire

1. le robot approche l'objet
2. ferme l'outil (gripper, pompe à vide, électromaimant..) pour saisir l'objet

3. se replace à quelques centimètres au-dessus de l'objet

Ce bloc est une ***condition*** (il est en forme de losange) et doit donc être inséré dans d'autres blocs (boucles, logique...).

Dans les blocs logiques, il existe un bloc prédéfini ***if-do-else***



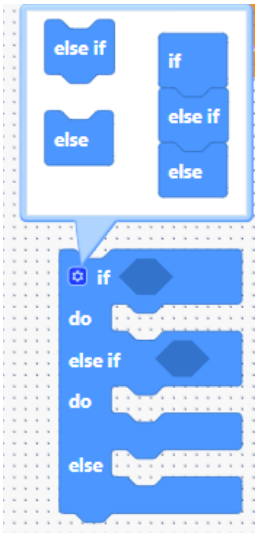
Ce bloc permet de

1. if: vérifier une condition
   Par exemple, si le robot est capable de faire le bloc "vision pick" = si les conditions du bloc sont remplis. (exemple: on demande au robot de saisir des objets rouges, sauf qu'il ne détecte que des objets verts => la condition n'est pas remplie)
2. do: si la condition est remplie, alors les instructions englobées dans le "do" seront exécutées
3. else: si la condition n'est pas remplie, alors les instructions suivantes seront exécutées.

Nous allons utiliser le bloc personnalisable:



En cliquant sur la roue crantée, on affiche les éléments qu'on peut ajouter au bloc et leur ordre. Par exemple:



Ici je peux dire au robot de vérifier une première condition ***if*** (et d'exécuter des instructions ***do***) avant de vérifier une deuxième condition ***else if*** (et d'exécuter des instructions ***do***) et enfin, de finir par des instructions ***else.***

Concrètement, nous allons donc dire au robot:

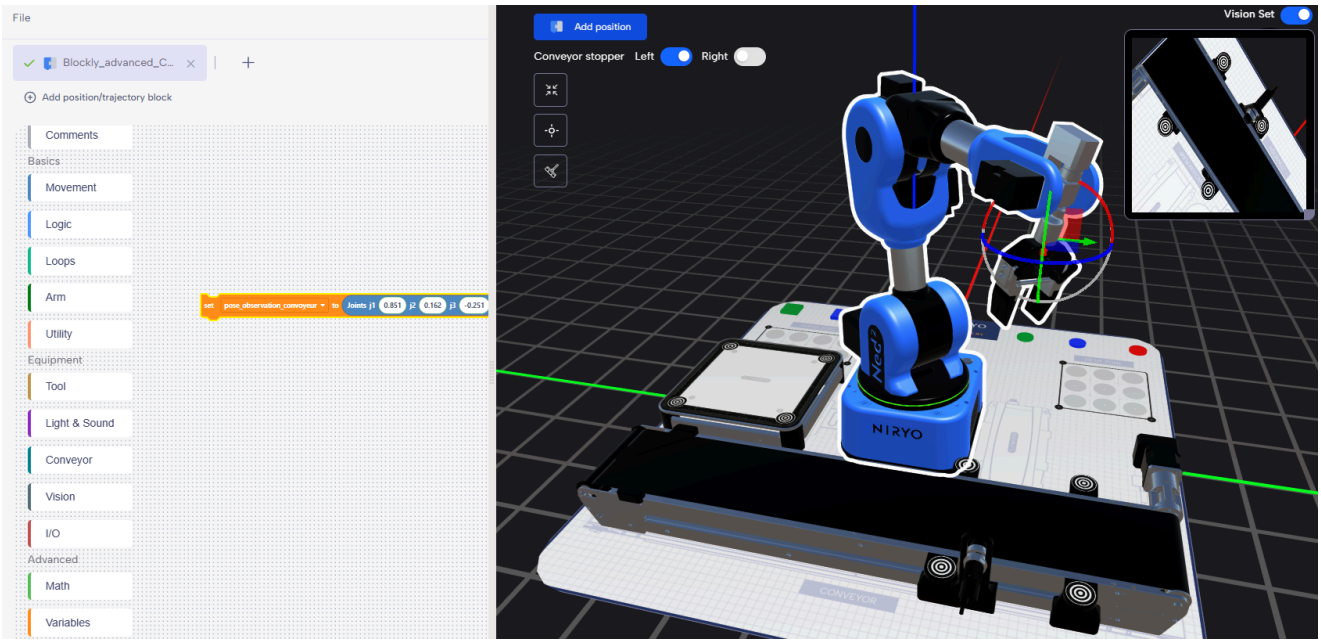
1. Si tu vois des jetons bleus --> alors place les dans la zone "plaque vision"
2. Si tu vois des jetons verts --> alors place les dans la zone "Alpha zone"
3. Sinon (jetons rouges) --> aucune action à effectuer.

J'utiliserai donc un bloc "if ...do" pour la première partie + "else if... do" pour la deuxième partie.

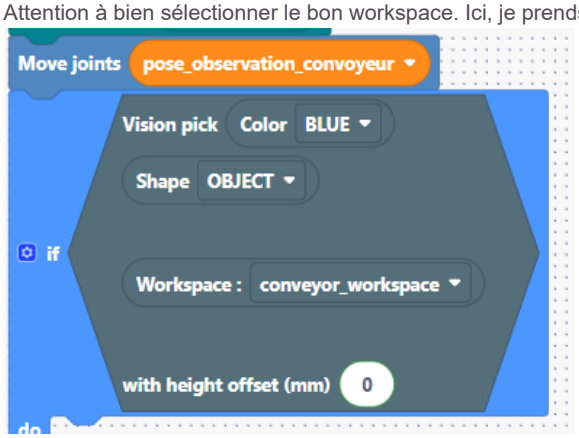
Résolution de la partie 2:

1. Je configure mon bloc condition personnalisé

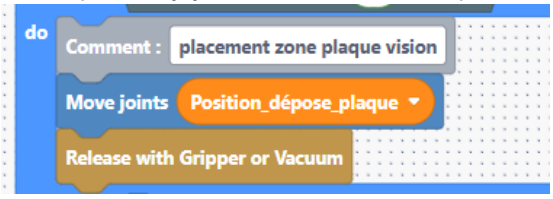
2. Je déplace mon robot dans une pose d'observation depuis laquelle il peut observer les 4 cibles du workspace du convoyeur. J'enregistre cette position dans une variable.



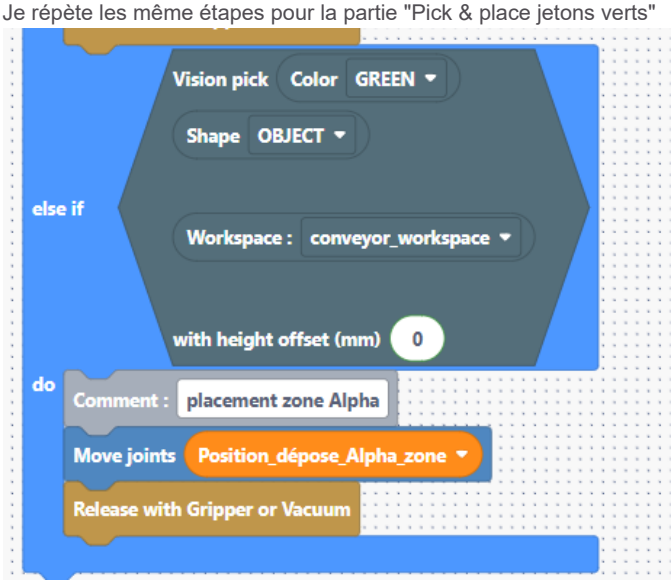
3. J'ajoute mon bloc ***Vision pick*** que je paramètre avec les conditions "bleu"
   Attention à bien sélectionner le bon workspace. Ici, je prends le *workspace du convoyeur*



4. Dans la partie ***do*** j'ajoute les éléments de la séquence "Déposer le jeton bleu détecté dans la zone plaque vision"

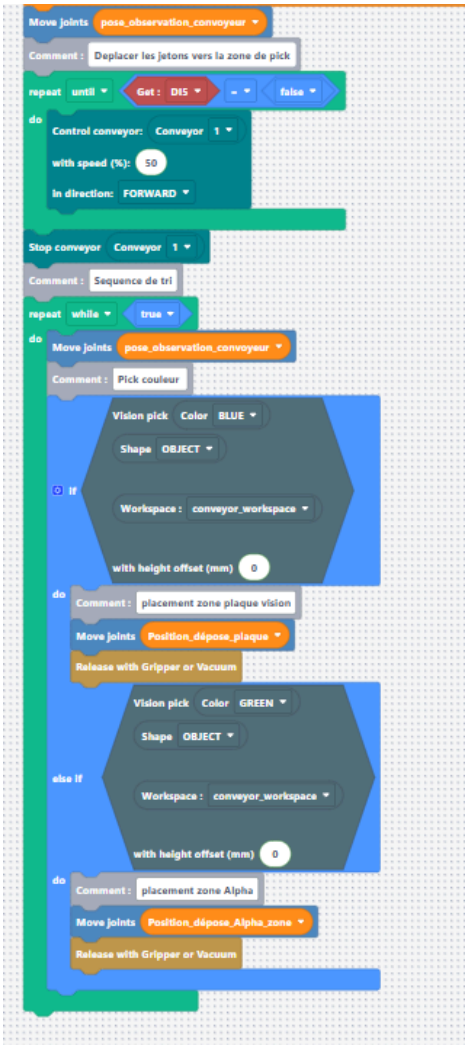


5. Je répète les même étapes pour la partie "Pick & place jetons verts"



6. Si je souhaite qu'on répète cette opération pour tous les jetons qui défilent sur le convoyeur on peut ajouter une boucle

7. Pour charger le convoyeur en amont du workspace, je saisis et place des jetons verts et des jetons bleus en amont du convoyeur.



Télécharger un exemple de solution:

🔗 Lien de téléchargement

💡 Vous pouvez importer ce programme directement dans NiryoStudio depuis Librairie > Import > fichier .json

Fonctions

Concept

Les **fonctions** permettent de regrouper du code réutilisable pour améliorer lisibilité et maintenance.

Cas pratique:

Partie 1

Nous allons reprendre la séquence de pick & place que nous avons créée dans le chapitre "**Variables**" et voir comment transformer une partie du programme avec les fonctions.

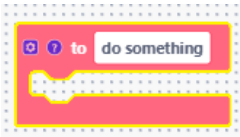
Pour rappel, voici notre programme:



Je vais désormais regrouper mes 3 séquences de pick & place dans 1 seule fonction.

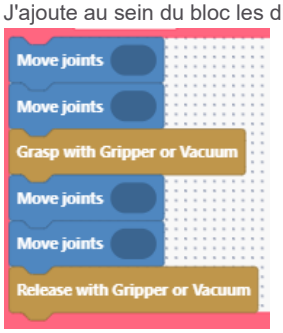
Résolution de la partie 1:

1. J'utilise le bloc ***Fonction*** que je personnaliserai ensuite

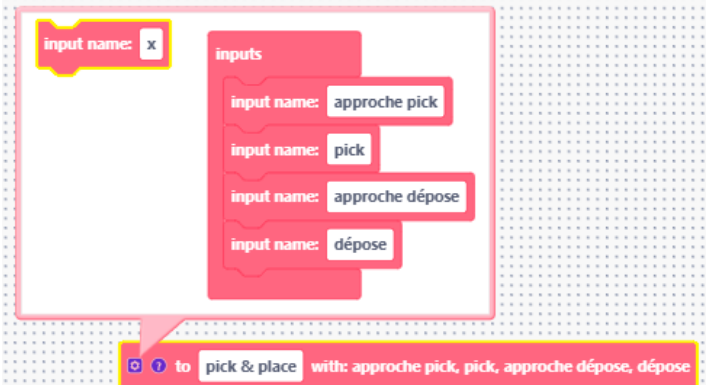


2. A la place de "**Do something**" je note le nom de ma fonction. Ici par exemple, je l'appelle "**Pick & Place**"

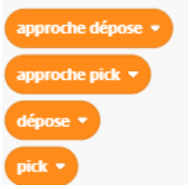
3. J'ajoute au sein du bloc les différentes instructions de ma séquence. Je laisse pour le moment les objets de les blocs "move joints" vides



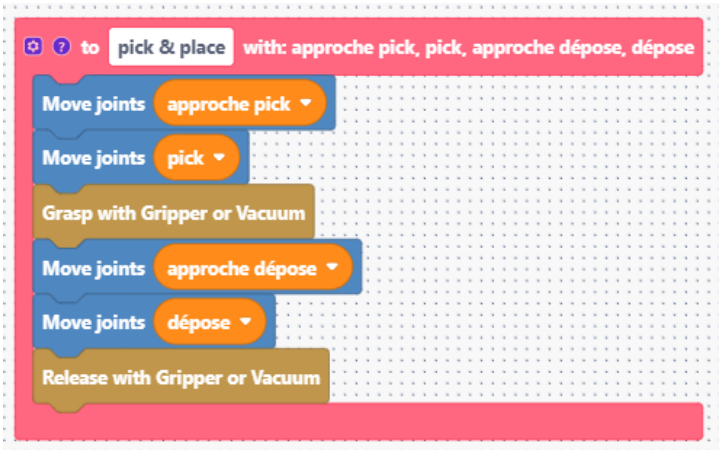
4. Je clique sur la *roue crantée*. C'est ici que je peux choisir mes variables de la fonctions, c'est à dire, ce que je peux modifier dans la structure de ma séquence (par ex. les coordonnées de mes positions).



J'en ajoute 4: approche pick, pick, approche dépose et dépose. En créant ces "input name", je crée en fait des blocs que je retrouves dans les ***variables***.

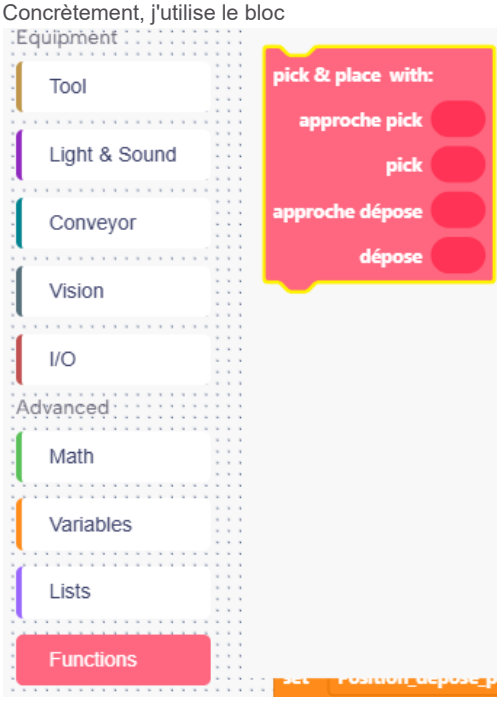


5. Je vais maintenant compléter ma séquence dans ma fonction avec les variables.



6. Voilà ! J'ai réalisé ma première fonction qui me permet de réutiliser une séquence simple de pick & place.

7. Concrètement, j'utilise le bloc



8. J'y ajoute les variables nécessaires à l'exécution de ma fonction:



Télécharger un exemple de solution:

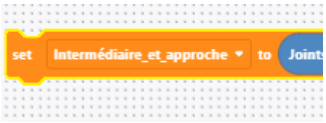
🔗 Lien de téléchargement

💡 Vous pouvez importer ce programme directement dans NiryoStudio depuis Librairie > Import > fichier .json

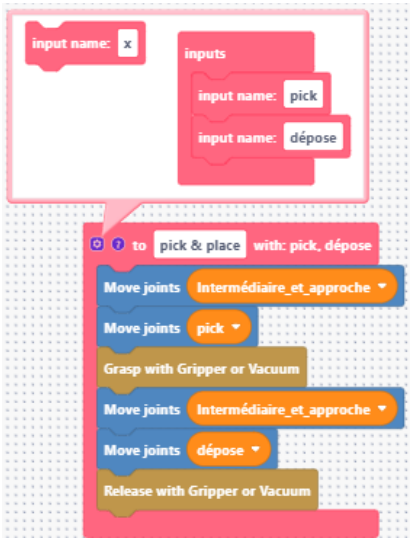
---

Je peux simplifier encore plus mes fonctions, en créant des positions d'approche et des positions intermédiaires de dépose unique entre l couleurs et identiques entre elles.

1. Nouvelle variable:



2. Fonction simplifiée (seulement 2 inputs pour la fonction):



3. Utilisation des fonctions:



Télécharger un exemple de solution:

🔗 Lien de téléchargement

💡 Vous pouvez importer ce programme directement dans NiryoStudio depuis Librairie > Import > fichier .json

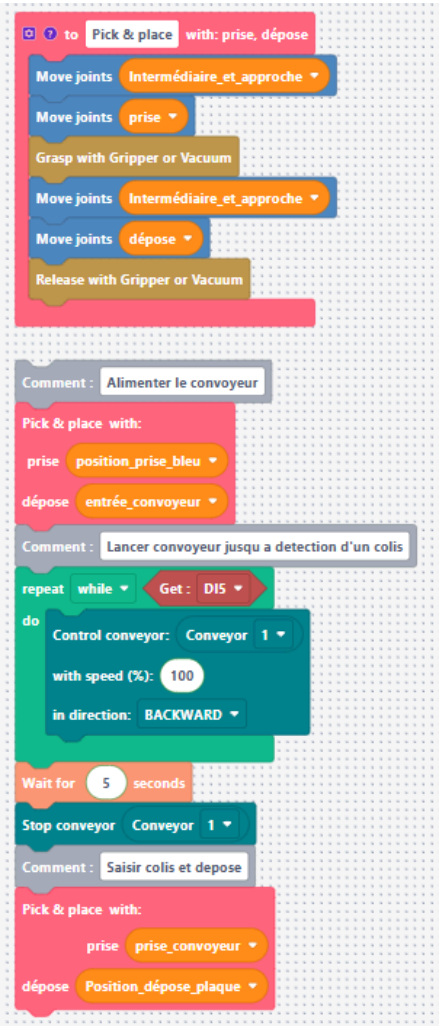
---

Partie 2

Nous allons reprendre la séquence de pick & place que nous avons créée dans le chapitre "**Boucles**" et voir comment transformer une partie du programme avec les fonctions.

Est-ce que vous pouvez faire une proposition d'optimisation de ce programme à l'aide des ***Fonctions***?

Résolution de la partie 2



Télécharger un exemple de solution:

🔗 Lien de téléchargement

💡 Vous pouvez importer ce programme directement dans NiryoStudio depuis Librairie > Import > fichier .json

---

Partie 3

Nous allons reprendre la séquence de pick & place que nous avons créée dans le chapitre "**Variables - Partie 1**" et voir comment transformer une partie du programme avec les fonctions.

Dans ce cas pratique, nous allons utiliser les ***Fonctions avec un output***. Ces fonctions permettent de retourner une valeur à la suite de la séquence qu'elle exécutent. C'est optionnel, on peut aussi simplement créer une fonction, sans séquence d'instruction.

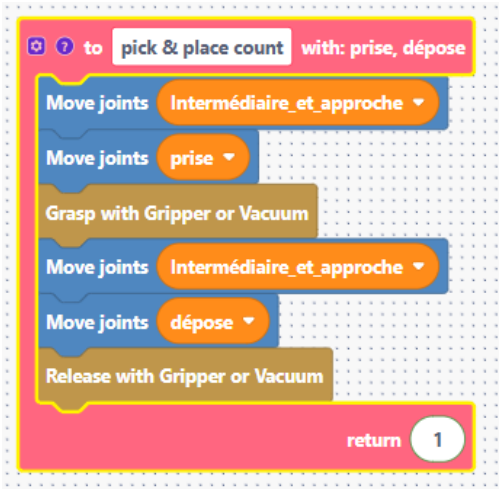


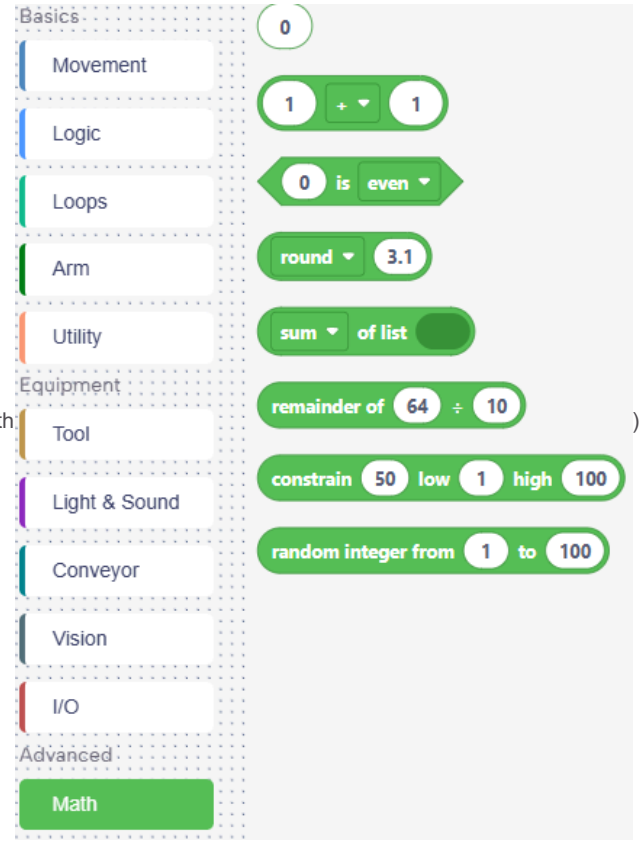
Ce type de fonction se paramètre de la même manière que les fonctions sans output.

Vous pouvez ensuite ajouter une valeur dans la partie ***return ...***.

Résolution de la partie 3

1. Je crée ma fonction avec output (idem que dans les parties 1 et 2):



En "output", j'ajoute un bloc (dans Math 

) qui renvoie "1"

Pourquoi? Cette fonction me permettra de compter le nombre de fois que je l'exécute. A chaque itération de cette fonction, elle donne "1", valeur que je peux utiliser pour incrémenter mon compteur.

2. Je peux maintenant utiliser cette fonction comme un ***objet*** utilisé au sein d'un autre bloc.

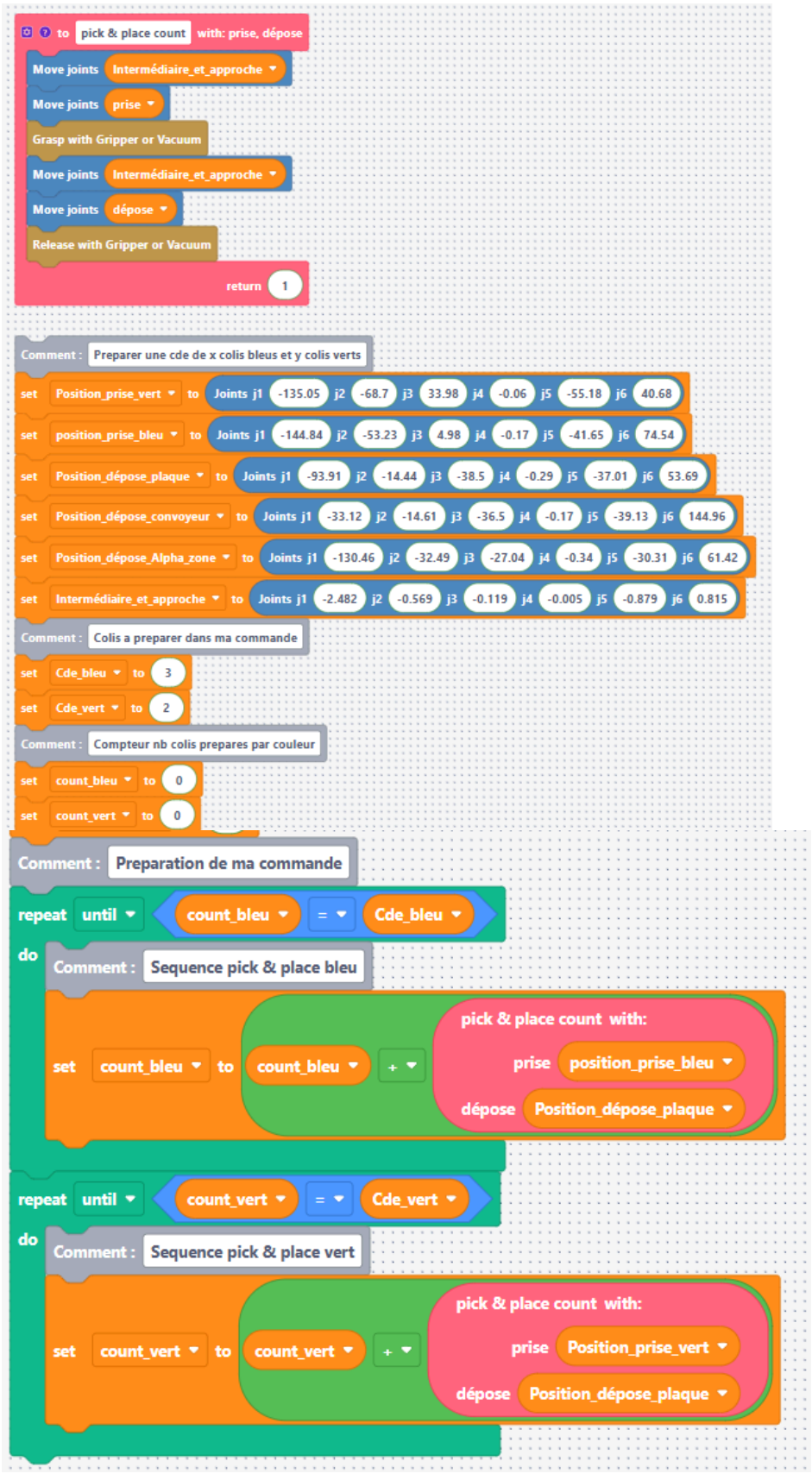


Explication: Dans mon programme précédent, j'exécutais ma séquence de pick & place, et ensuite, j'incrémentais un compteur en disant: "nombre de colis bleus déjà préparés" + 1.
Ici, je remplace "+1" par "+ ***Fonction avec output***" avec les paramètres nécessaires à son exécution.
Le programme va donc se dérouler ainsi:

1. La fonction s'exécute (les instruction du pick & place --> la valeur 1 est retournée"

2. Je définis la variable (***set ...***) "count_bleu" en incrémentant sa propre valeur + la valeur de la fonction "pick & place count" (qui est 1).

3. Résultat final:



Télécharger un exemple de solution:

🔗 Lien de téléchargement

💡 Vous pouvez importer ce programme directement dans NiryoStudio depuis Librairie > Import > fichier .json

Listes

Fonctions

Concept

Les **listes** sont un conteneur qui peuvent stocker plusieurs valeurs dans une seule variable.

Cas pratique:

Nous allons nous inspirer de la séquence de pick & place que nous avons créée dans le chapitre "**Boucles - Partie 1**" et voir comment transformer une partie du programme avec les listes.

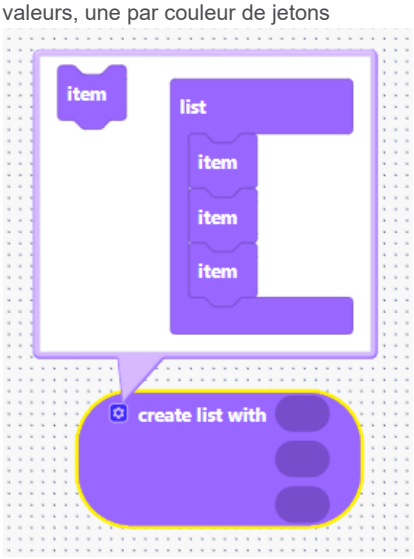
Cette fois-ci, on souhaite dire au robot de prendre 1 jeton de chaque couleur et de le placer sur la zone de dépose "Plaque vision"

🚫 On pourrait vouloir faire 3 séquences avec 1 séquence par jeton de couleur.

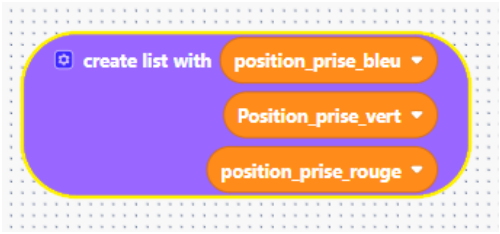
Au lieu de cela, nous allons utiliser le bloc ***Listes***.

Résolution:

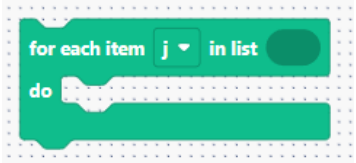
1. Sélectionnez le bloc "Listes" et avec la roue crantée, sélectionnez le nombre de valeurs à stocker dans la liste. Ici, on a besoin de 3 valeurs, une par couleur de jetons



2. On y stocke nos variables de positions de prise des jetons:

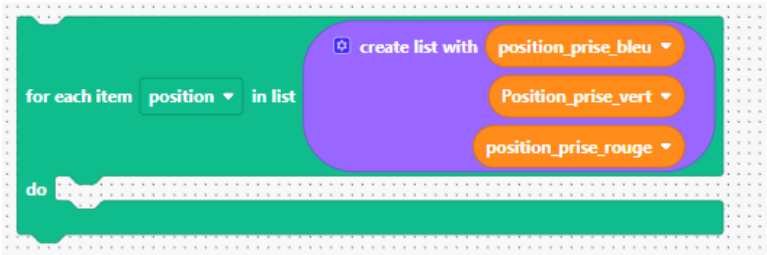


3. Pour dire au robot "passe automatiquement par toutes les positions de la liste pour effectuer une séquence", on intègre cette liste dans une boucle. Cette boucle est prédéfinie:



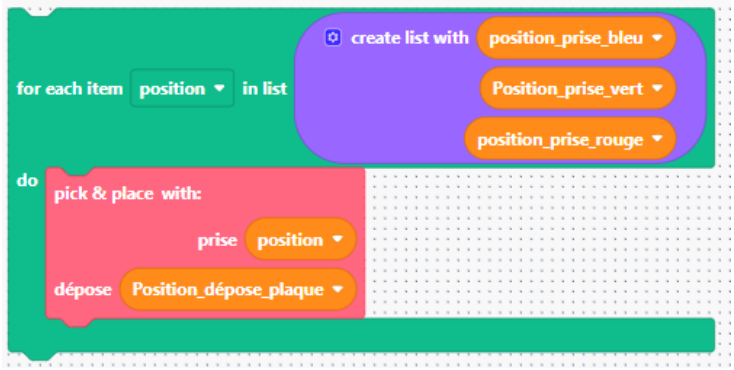
Qui dit: pour chaque item (ici variable j que l'on peut renommer "position") dans la liste ... ***do*** (exécute) ces instructions

4. On intègre la liste à cette boucle:



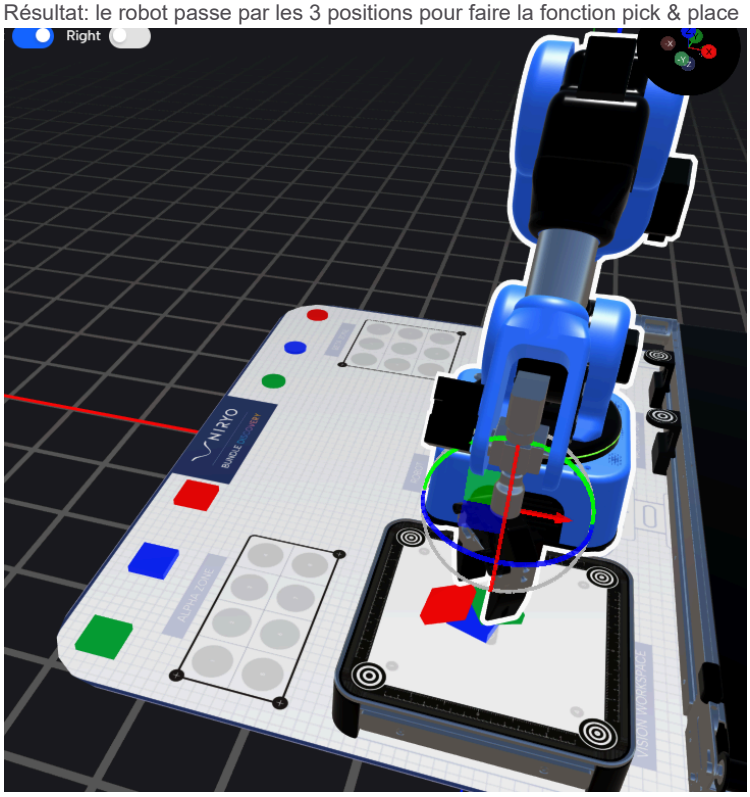
5. Puis on ajoute une séquence de pick & place (soit une suite d'instructions, soit une fonction). Je vous montre ici l'exemple avec la fonction:





Explication: Ma boucle dit "pour chaque item "position" de la liste, exécute la fonction". La fonction reprend la valeur de la variable "position" (soit les positions stockées dans la liste) pour la position de prise.

6. Résultat: le robot passe par les 3 positions pour faire la fonction pick & place



Télécharger un exemple de solution:

🔗 Lien de téléchargement

💡 Vous pouvez importer ce programme directement dans NiryoStudio depuis Librairie > Import > fichier .json